CASE STUDY 161

Random Password Generator using C++

Name: Aashika Pandey
Roll No:150096725010
Course: B.Tech CSE 2025-29
Subject: C++ Programming (Semester II)
University: ITM Skills University

---

1. Overview

This case study focuses on developing a **Random Password Generator using C++**. The program generates secure passwords using a combination of uppercase letters, lowercase letters, numbers, and special symbols. The user can define the desired password length, and the system will generate a random password accordingly.

The purpose of this project is to understand **random number generation, string manipulation, and basic security concepts** in C++.

---

2. Objectives

The objectives of this case study are:

- To understand the concept of **random number generation** in C++.
- To apply **string manipulation techniques**.
- To generate secure passwords using different types of characters.
- To promote **basic security awareness** by using strong passwords

---

3. Requirements

Password Rules

The generated password should follow these rules:

- The password length should be **fixed or user-defined**.
- The password should contain a **combination of characters**, including:
    - Uppercase letters (A-Z)
    - Lowercase letters (a-z)
    - Numbers (0-9)
    - Special symbols (@, #, $, %, &, *)

Operations

The program should perform the following operations:

1. Generate a random password.
2. Display the generated password to the user.

---

4. Program Flow

The program works in the following steps:

1. Display a menu with options:
    - Generate Password
    - Exit

2. Accept the user's choice.
3. If the user selects **Generate Password**, the program asks for password length.
4. The program generates random characters using predefined character sets.
5. A password is created by combining uppercase letters, lowercase letters, numbers, and symbols.
6. The generated password is displayed on the screen.
7. The menu appears again until the user chooses to exit.

---

5. Algorithm

Step 1: Start the program.
Step 2: Define character sets (uppercase, lowercase, numbers, symbols).
Step 3: Combine all characters into one string.
Step 4: Initialize random number generator using `srand(time(0))`.
Step 5: Display menu options.
Step 6: Take user input for choice.

If choice = 1

- Ask user for password length.
- Ensure password length is at least 4.
- Add one uppercase letter randomly.
- Add one lowercase letter randomly.
- Add one number randomly.
- Add one symbol randomly.
- Fill remaining characters randomly from the combined set.
- Display the generated password.

If choice = 2

- Exit the program.

Step 7: End the program.

---

6. C++ Program Code

randompass > random.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

int main()
{
    int choice, length;

    string upper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    string lower = "abcdefghijklmnopqrstuvwxyz";
    string numbers = "0123456789";
    string symbols = "@#$%&*";

    string all = upper + lower + numbers + symbols;

    srand(time(0));

    while (true)
    {
        cout << "\n----- Random Password Generator -----\n";
        cout << "1. Generate Password\n";
        cout << "2. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter password length: ";
            cin >> length;

            if (length < 4)
            {
                cout << "Password length should be at least 4\n";
                continue;
            }

            string password = "";

            // Ensure one of each type
            password += upper[rand() % upper.length()];
            password += lower[rand() % lower.length()];
            password += numbers[rand() % numbers.length()];
            password += symbols[rand() % symbols.length()];

            for (int i = 4; i < length; i++)
            {
                password += all[rand() % all.length()];
            }

            cout << "Generated Password: " << password << endl;
        }
        else if (choice == 2)
        {
            cout << "Exiting program...\n";
            break;
        }
        else
        {
            cout << "Invalid choice. Try again.\n";
        }
    }

    return 0;
}
```

---

7. Sample Output

Example output of the program:

----- Random Password Generator -----

1. Generate Password
2. Exit
Enter your choice: 1
Enter password length: 8
Generated Password: Px9$RYDi

```
----- Random Password Generator -----
1. Generate Password
2. Exit
Enter your choice: 1
Enter password length: 8
Generated Password: Px9$RYDi
```

---

8. Learning Outcomes

From this case study, the following concepts were learned:

- Randomization logic in C++
- Generating secure passwords
- Use of strings in C++
- Implementation of menu-driven programs
- Importance of strong passwords in security

---

9. Conclusion

The Random Password Generator program successfully generates secure passwords using a combination of uppercase letters, lowercase letters, numbers, and symbols. The program demonstrates the use of random number generation and string manipulation in C++. It also highlights the importance of creating strong passwords for better security.

---

10. References

1. Bjarne Stroustrup, **The C++ Programming Language**, Addison-Wesley.
2. C++ Reference Website – https://cplusplus.com

3. GeeksforGeeks – https://www.geeksforgeeks.org (for understanding random number generation in C++).
4. ITM Skills University – **Case Study 161: Random Password Generator (C++) Problem Statement**.

---

10. GiHub Link

CASE STUDY 161